

IDEA → APPLICATION

From Idea to Application

Three ways to get there

Old-school coding, Streamlit's mid-code prototyping, and vibe coding with Google AI Studio, same destination, wildly different roads.



Hamid Bekamiri

Session 4 · BDS M1

Two hours of ideas, two hours of building

HOURS 1-2 · LECTURE AND MINI PROJECT

- Three eras of building software — and what each one costs you.
- Foundations: machines, servers, APIs, environments.
- Mini project: build the same small app twice. Once in Python, once by describing it in English.
- You follow a worksheet. The answer is already known.

HOURS 3-4 · WORKSHOP

- Your group picks a question worth answering.
- Real data from a free API — no key, no sign-up.
- 45 minutes to build something that runs.
- Then 5 minutes to convince the room.
- Nobody hands you the answer.

Learning outcomes

By the end of this session you will be able to:

- 1 Trace how the tools for turning an idea into a working app have changed! From punch cards to Streamlit to vibe coding
- 2 Compare old-code, mid-code (Streamlit) and vibe-coding approaches on effort, skill required, and how much you actually control.
- 3 Build a small interactive app twice: once in Streamlit (Python), once by prompting Google AI Studio.
- 4 Use caching (`@st.cache_data`) and session state to keep a Streamlit PoC fast and stateful.
- 5 Critically weigh what each approach hides from you and when that trade-off is worth making.

Why move beyond static figures?

Storytelling with data lives or dies on comparison and context. A static chart fixes both for you — an app lets the audience set them.

THE GAP A STATIC CHART LEAVES

- Faster feedback: stakeholders explore scenarios and ask what-if questions live.
- Bridging the gap: wrap the analysis in a simple UI — no notebook required to use it.
- Narrative + exploration: guided story, then self-service digging.

WHEN A DATA APP EARNS ITS KEEP

- You need interactivity (filters, parameters) or scenario testing.
- You want something reusable that others open in a browser.
- Scope is small and focused — a PoC, not a product.

Milad Abb + 9 • 17h

BDS visualization

Milad Abb

19 hours ago

Dataset variables

owid/co2-data

Data on CO2 and greenhouse gas emissions by Our World in Data

github.com

co2-data/owid-co2-codebook.csv at master · owid/co2-data

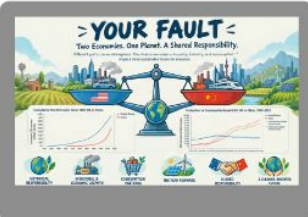
Add comment

0

Anonymous

17 hours ago

Climate Policy Responsibility



Responsibility debate - for a climate summit between China and the US (for young professionals)

Add comment

0

Table in front of teacher

17 hours ago

Same planet, same scale



u1.padletusercontent.com

same-planet-same-scale

The change in time shows a change in conditions. The world united and implemented initiatives against Co2 emissions, which we wanted to see if they held true.

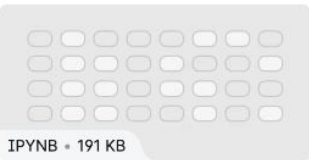
Add comment

2

Anonymous

17 hours ago

Co2 production per trade, gdp and capita



IPYNB • 191 KB

co2_china_us_and_denmark

Add comment

0

anonymous

17 hours ago

The ozempic of penguins



10

10

10

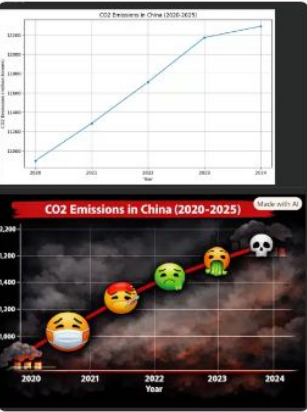
Add comment

0

Farvefinder Paletten

17 hours ago

CHINA BAD



1

1

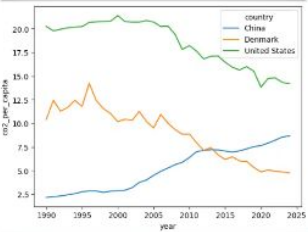
1

Add comment

0

Anonymous

17 hours ago



Add comment

0

-

17 hours ago

CO2 and GDP per capita in Europe



u1.padletusercontent.com

co2_gdp_europe_slides

Add comment

0

Ariel

17 hours ago

Can economies grow without increasing CO2 emissions?

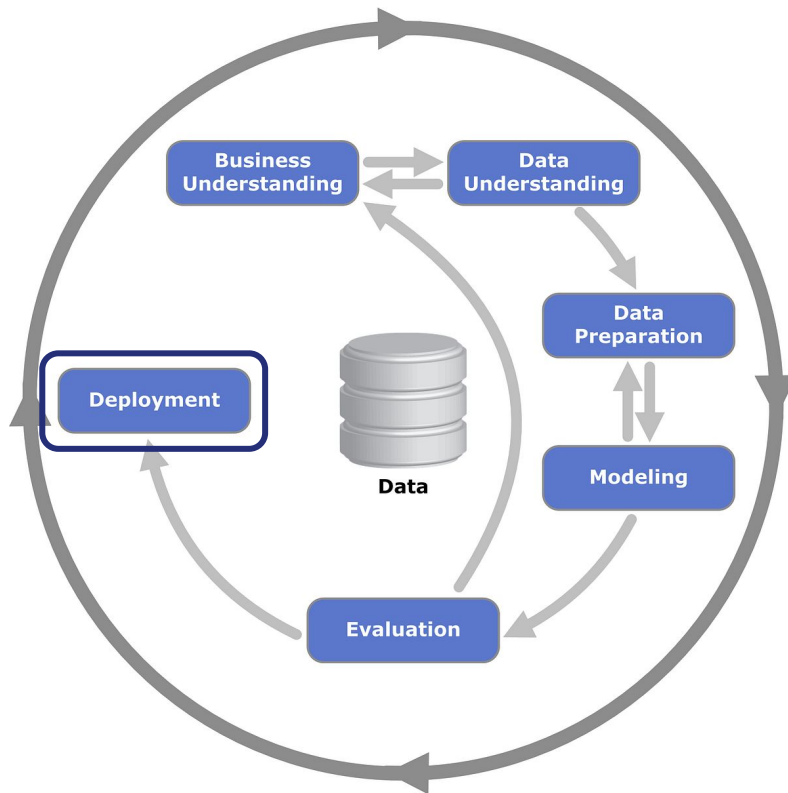


Audience: Business executives
By: Ariel Martinelli, Pablo Piccioni and Fernando Perales.

Add comment

0

CRISP-DM's last box is this entire session



BOXES 1-5 → BOX 6

- **Runs on** — your laptop → **a server that is always on**
- **Built for** — you → **somebody else**
- **What you produce** — a finding, in a notebook → **a thing someone opens**

Data visualization toolkit

3) Know what to visualize

SPRINGER NATURE Link Login

Find a journal Publish with us Track your research Saved research

Home > Genome Biology > Article

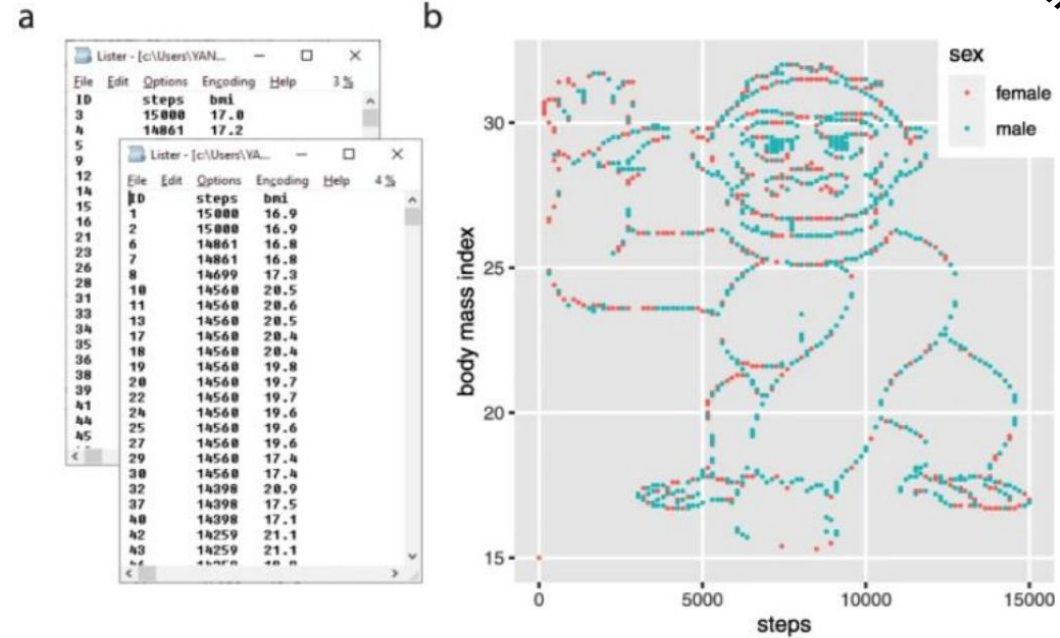
A hypothesis is a liability

Editorial | [Open access](#) | Published: 03 September 2020
Volume 21, article number 231 (2020) [Cite this article](#)

☒ You have full access to this open access article

[Download PDF](#) [Save article](#)

Fig. 1



Gorilla in the data

c

	Gorilla <u>not</u> discovered	Gorilla discovered
Hypothesis-focused	14	5
Hypothesis-free	5	9

The hidden cost of having a hypothesis

Scientific progress relies on two complementary modes. While day science tests focused ideas, night science explores raw data for unexpected patterns.

DAY SCIENCE · TESTING THE HYPOTHESIS

- **Strengths:** Guides test design, parameter estimation, and rigorous verification.
- **The Liability:** Mental focus on a specific hypothesis creates tunnel vision, blinding analysts to other unexpected discoveries in rich datasets.
- **Impact:** Limits visual data exploration and creativity in complex biological data.

NIGHT SCIENCE · EXPLORING THE UNKNOWN

- **Strengths:** Unconstrained exploration, generating novel ideas and uncovering hidden patterns in vast datasets.
- **The Liability:** Frequently generates spurious relationships and false hypotheses.
- **Resolution:** Exposed and filtered by day science, making both modes strictly essential and complementary.

The basics — what stuck?

No names, no grades. Just so I know where to slow down



Join at
slido.com
#2888 216

A computer, and two of them talking

WHAT IS ACTUALLY INSIDE ONE COMPUTER

- CPU — does the work. One thing at a time, but extremely fast.
- RAM — the desk. Whatever is being worked on right now. Emptied the moment the power goes.
- Disk — the filing cabinet. Slower, but it still remembers tomorrow.
- GPU — an optional second worker, for thousands of identical sums at once.
- Network card — the only way anything gets in or out.

WHAT HAPPENS WHEN TWO OF THEM TALK

- The client asks. The server answers. That is the entire idea.
- Your browser is a client. So is a phone app — and so is your Python script the moment it calls an API.
- A server is the same box, with the same CPU, RAM and disk. Someone simply left it switched on and gave it a public address.
- Every website you open: a request goes out, a response comes back. Nothing more.
- “The cloud” is renting somebody else's box by the hour instead of buying your own.

CPU or GPU: one expert chef, or a thousand line cooks

WHAT THE CHIP ACTUALLY DOES

- A CPU has a handful of fast, flexible cores — one expert chef who can cook anything, one dish at a time.
- A GPU has thousands of simple cores doing the same operation at once — a thousand line cooks, all chopping.
- Training a neural network is mostly one operation repeated billions of times: multiply a matrix. That is precisely what a GPU is built for.
- RAM is the counter space. If the data does not fit in it, nothing runs — however fast the chip is.



Where does the code actually run?

LAPTOP, SERVER, CLOUD

- A server is just a computer that is always on and has a public address. There is nothing more magical about it than that.
- “The cloud” means renting one of those by the hour instead of buying it and putting it in a cupboard.
- Your laptop is a perfectly good computer. It is simply asleep, shut, or on a train about half the time.
- Deploying = copying your code onto an always-on machine and getting a URL back in exchange.

WHERE YOU MEET IT TODAY

- `http://localhost:8501`— “localhost” means this machine. Nobody else on earth can open that link.
- `https://yourname.streamlit.app`— same script, someone else's always-on computer, anyone can open it.



EC2 M5 Instance Specifications & Performance

GENERAL PURPOSE M5 & M5D SPECIFICATIONS

Instance Size	vCPU	Memory (GiB)	Instance Storage (GiB)	Network Bandwidth	EBS Bandwidth
m5.large	2	8	EBS-Only	Up to 10 Gbps	Up to 4,750 Mbps
m5.xlarge	4	16	EBS-Only	Up to 10 Gbps	Up to 4,750 Mbps
m5.2xlarge	8	32	EBS-Only	Up to 10 Gbps	Up to 4,750 Mbps
m5.4xlarge	16	64	EBS-Only	Up to 10 Gbps	4,750 Mbps
m5.8xlarge	32	128	EBS-Only	10 Gbps	6,800 Mbps
m5.12xlarge	48	192	EBS-Only	12 Gbps	9,500 Mbps
m5d.large	2	8	1 x 75 NVMe SSD	Up to 10 Gbps	Up to 4,750 Mbps
m5d.xlarge	4	16	1 x 150 NVMe SSD	Up to 10 Gbps	Up to 4,750 Mbps
m5d.4xlarge	16	64	2 x 300 NVMe SSD	Up to 10 Gbps	4,750 Mbps
m5d.12xlarge	48	192	2 x 900 NVMe SSD	12 Gbps	9,500 Mbps
m5d.16xlarge	64	256	4 x 600 NVMe SSD	20 Gbps	13,600 Mbps

Higher instance sizes scale compute, memory, and bandwidth linearly

An API rents you a capability you will never own

WHAT YOU ARE ACTUALLY BUYING

- An API is a fixed way to ask another company's software to do something, and get structured data (JSON) back.
- You are not buying the model, the data, or the machines. You are buying the answer, one question at a time.
- The key is your name on the bill. Whoever holds it spends your money — treat it as a credit card, not a username.
- Two limits always apply: how often you may ask (rate) and what each ask costs (per call, or per 1,000 tokens).

WHERE YOU MEET IT TODAY

- `client.models.generate_content(...)` — one line of yours, Google's GPUs, your invoice.
- `secrets.toml` — where Streamlit Cloud keeps your keys (Advanced settings). Never inside `app.py`.
- A key pasted into code is a key in your git history — permanently, even after you delete the line.
- The March 2025 story later in this deck begins with API keys that nobody was watching.

APIs: how the pieces actually talk

Every era we're covering eventually calls an API — either one you write, or one someone else runs for you.

YOUR OWN API (ERA 1'S MISSING PIECE)

- An API is just a URL your frontend can send a request to and get structured data (usually JSON) back.
- GET /students — list of records. POST /students — create one. The URL + method is the “endpoint.”
- This is the “an API” in Era 1's own list — Flask/Express turn your functions into exactly these routes.

SOMEONE ELSE'S API (ANY ERA)

Calling a third-party API is the same idea, just aimed outward — e.g. the Gemini API:

```
from google import genai
client = genai.Client(api_key= "...")
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="Why is the sky blue?" )
```

Same shape either way: send a request, get data back. You just don't own or run that server.

Why “it works on my machine” is not a joke

YOUR CODE IS NEVER ALONE

- Your 70 lines sit on top of dozens of libraries, each with a version number that can change underneath you.
- A virtual environment is a separate cupboard per project, so one project's pandas cannot break another's.
- requirements.txt is the shopping list that lets anyone — or any server — rebuild that cupboard exactly.
- This is reproducibility: the same habit that separates a result from an anecdote in your thesis.

WHERE YOU MEET IT TODAY

- `python -m venv .venv`— a cupboard for this project only.
- `pip install streamlit pandas plotly`— fills it.
- `pip freeze > requirements.txt`— writes the list down.
- Streamlit Cloud reads that file and rebuilds the cupboard on its own machine. No file, no app.

Idea to application is four stages, not one step

Each stage answers a different question. Today's session lives entirely in stage two — and stage two is meant to be thrown away.

Stage	The question it answers	Who builds it	How long	What happens to it afterwards
1 • Idea	Is this worth building at all?	You, a pen, and some conversations	Hours	Binned — you keep the decision, not the sketch
2 • Prototype	What would it actually feel like?	One person, moving fast	Hours to days	Binned — you keep what you learned
3 • MVP	Will real people use it for real work?	One or two, carefully	Weeks	Kept, but expect to rewrite half of it
4 • Product	Can a business depend on it?	A team, plus someone to answer the phone	Months, then forever	Kept, and paid for forever

Where do today's tools sit in the prototyping spectrum?

“Idea to app” isn't one path — it's a ladder of options, each trading effort for realism.

Approach	Time to first version	Best for
Sketches & paper	Minutes	Validating the flow before any code exists
Clickable mockups (Figma)	Hours	Testing UX & stakeholder buy-in — nothing computes
No-code builders (Bubble, Glide)	Hours–days	A real functioning app, boxed in by the tool
Notebooks (Jupyter/Colab)	Hours	Exploring data/models fast — not shareable as an app
Streamlit — today's Era 2	Hours	Real Python code + real data + a shareable UI
Vibe coding — today's Era 3	Minutes	Describe it in English, an agent builds it
Full custom dev	Weeks+	Production-grade, full control

A brief history of building software

Each decade moved a little more of the “how” off your fingers and onto a tool.

- 1950s** ● **Machine code & punch cards** Every instruction written by hand, one bit at a time.
- 1970s** ● **High-level languages (C, Pascal)** A compiler translates readable code into machine code for you.
- 1990s** ● **Visual tools (VB, Delphi)** Drag components onto a form — still real programming underneath.
- 2000s** ● **Web frameworks (Rails, Django, PHP)** A website in days, not months.
- 2010s** ● **Cloud & no-code (AWS, Bubble)** Infrastructure becomes someone else's problem.
- Late 2010s** ● **Mid-code prototyping (Streamlit)** Pure Python; the framework builds the UI — today's Era 2.
- Early 2020s** ● **AI pair programming (Copilot)** AI suggests the next line as you type — you're still the author.
- 2025–** ● **Vibe coding (Google AI Studio)** Describe it in English; an agent writes and runs the whole app — today's Era 3.

Punch Card in Punch Card Machine



ENIAC: the world's first programmable electronic computer



Three eras, one destination

Every era answers the same question — with a completely different amount of typing.

01

The old approach

HAND-CODED

Write every line yourself: HTML/CSS/JS, a backend, a database, a server to run it on.

Needs a trained developer

02

Streamlit

MID-CODE · SINCE 2019

Write Python only. The framework turns your script into a reactive web UI.

Needs someone who can code

03

Vibe coding

GOOGLE AI STUDIO · 2025-

Describe the app in plain English. An AI agent writes the code for you.

Needs a clear description

Hand-coded: full control, full effort

WHAT IT TAKES

- Frontend: HTML, CSS, JavaScript (or a framework on top)
- Backend: a language, a web framework, routing, an API
- A database schema you design and migrate yourself
- Hosting, deployment, monitoring — the DevOps tax

TRADE-OFF

- Strength: nothing is a black box — you own every decision.
- Weakness: days to weeks before a stakeholder sees anything — expensive way to test whether an idea is even worth building.

Python only. Still code, just far less of it.

WHAT IT TAKES

- A single Python script, no HTML/CSS/JS required
- A reactive rerun model: the script reruns top to bottom on every interaction
- `@st.cache_data` and `st.session_state` to stay fast and stateful

TRADE-OFF

- Strength: hours, not weeks — and every line is Python you (or a teammate) can read and debug.
- Weakness: still requires programming skill, and you're limited to what Streamlit's widgets support.

Streamlit: local setup & first run

A fresh, isolated environment (Streamlit's own recommendation)

```
# macOS / Linux
python -m venv .venv && source .venv/bin/activate

# Windows (PowerShell)
python -m venv .venv; .venv\Scripts\Activate.ps1

pip install streamlit pandas plotly
```

Run it

```
streamlit run app.py
```

https://github.com/HamidBekamiri/BDS-Teaching/blob/main/Slides/Prototype/Track_A_-_Streamlit_-_Watchlist_Explorer.pdf

Streamlit: from script to public URL

No server to manage — Streamlit Community Cloud builds and hosts the app for you, for free.

- 1 **Push it to GitHub** — commit `app.py` and a `requirements.txt` to a repo.
- 2 **Sign in** — at share.streamlit.io with your GitHub account.
- 3 **Click “Create app”** — → “Yup, I have an app.”
- 4 **Point it at your repo** — branch and the path to `app.py` (or paste the GitHub URL).
- 5 **Advanced settings** — pick a Python version, paste your `secrets.toml` if the app needs API keys.
- 6 **Click Deploy** — live at [https://\[yourname\].streamlit.app](https://[yourname].streamlit.app), usually within minutes.

Never Commit API Keys to GitHub

Public bots scan repositories instantly, leading to stolen secrets, high bills, and compromised accounts.

THE RISK

- **Public Repos:** Automated scanner bots harvest exposed credentials in seconds after you push code.
- **Consequences:** Instant unauthorized access, rapid financial drain, and potential infrastructure compromise.
- **Private Repos:** Still considered a critical bad security practice due to risk of accidental leaks or broad access.

THE SOLUTION (BEST PRACTICE)

- Keep **secrets.toml** exclusively on your local machine inside the **.streamlit/** folder.
- Add **.streamlit/secrets.toml** to your **.gitignore** file so Git completely ignores it.
- When deploying, paste the secrets directly into the "**Advanced settings**" tab on Streamlit Cloud.
- Streamlit securely encrypts your keys and injects them privately at runtime.

Folder Structure

Plaintext

```
my-streamlit-app/  
├── .streamlit/  
│   └── secrets.toml    <-- ONLY on your local machine  
├── .gitignore  
├── requirements.txt  
└── app.py
```

1. requirements.txt

Tells the cloud server which libraries to install.

Plaintext

```
streamlit>=1.35.0  
openai>=1.30.0
```

2. .gitignore

Tells Git to never track or upload your secrets.

Plaintext

```
# Ignore local secrets  
.streamlit/secrets.toml  
  
# Ignore Python cache files  
__pycache__/  
*.pyc  
.env
```

3. Local .streamlit/secrets.toml

Lives only on your computer for testing.

Ini, TOML

```
OPENAI_API_KEY = "sk-proj-your-real-api-key-here"
```

4. app.py

Reads the secret cleanly through `st.secrets`.

Python

```
import streamlit as st  
from openai import OpenAI  
  
st.title("My Secure Streamlit App")  
  
# Streamlit automatically reads from secrets.toml (local or cloud)  
api_key = st.secrets["OPENAI_API_KEY"]  
client = OpenAI(api_key=api_key)  
  
user_prompt = st.text_input("Ask a question:", "Why is the sky blue?")  
  
if st.button("Submit"):  
    response = client.chat.completions.create(  
        model="gpt-4o-mini",  
        messages=[{"role": "user", "content": user_prompt}],  
    )  
    st.write(response.choices[0].message.content)
```

5. Git Commands to Push to GitHub

Run these in your terminal from your project folder:

Bash



```
# 1. Initialize the repo
git init

# 2. Add files (Git will automatically skip .streamlit/secrets.toml)
git add .

# 3. Check status to confirm secrets.toml is NOT listed under tracked files
git status

# 4. Commit and push
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
git push -u origin main
```

Summary of the Flow

- **On your computer:** `secrets.toml` lives on your hard drive → your local app reads `st.secrets`.
- **On GitHub:** Contains only code, no secret keys.
- **On Streamlit Cloud:** You paste the values into the web interface → the cloud app reads `st.secrets`. [Streamlit documentation](#)

45 minutes to build. Five minutes to convince.

THE 45 MINUTES

- 0-5 · Agree your question. One sentence, written down.
- 5-10 · Open the API in a browser. Look before you build.
- 10-35 · Build. One feature at a time.
- 35-40 · Check one number against the raw data, by hand.
- 40-45 · Prepare the pitch. Decide who says what.

THE 5-MINUTE PITCH

- The question — who asks it, and why they care. (30 sec)
- The data — what you got, and what was missing. (30 sec)
- Show it — live, not screenshots. (2 min)
- One thing you checked — how do you know it is right? (1 min)
- What would make it real, and what would that cost? (1 min)

Stop building at minute 35. Whatever runs then is what you show.

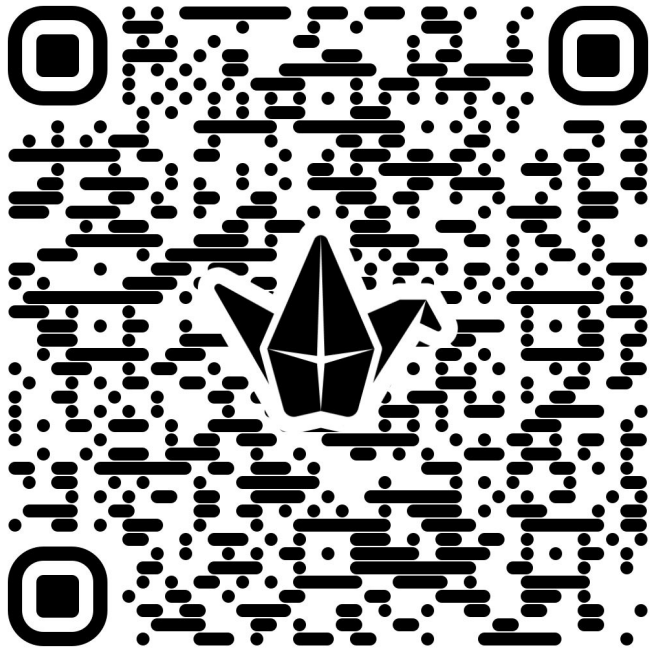
Where to hunt for an idea



THREE PLACES TO LOOK — AND HOW TO CHOOSE

- **Start here** — mixedanalytics.com, the list behind this code. No key, no sign-up, updated 2026.
- **Go wider** — github.com/public-apis/public-apis Filter the Auth column to “No”.
- **Browse by topic** — publicapis.io, 1,500 APIs across 47 categories.
- **Scroll past the toys.** Jokes, Pokémon and random dog pictures dominate. The useful ones are fewer.
- **Pick a question first.** Choosing an API and then hunting for a question is how groups lose twenty minutes.
- **Stuck after five minutes?** Use one of the four on slide 22 and start building.

Upload before you pitch



WHAT GOES ON THE PADLET

- The link to your live app — one that anyone can open.
- At most three slides. Fewer is better.
- One sentence: the question you set out to answer.
- Your names.
- Cannot scan it? The same link is on Moodle.

Describe it. An agent builds it.

WHAT IT TAKES

- A plain-English prompt, typed into Google AI Studio's Build tab
- Google's Antigravity agent writes HTML/CSS/TypeScript for a simple app, or a full React + Node.js app for anything more complex
- A live preview updates as you keep chatting

TRADE-OFF

- Strength: minutes, not hours — no programming skill required to get something clickable.
- Weakness: you often don't know why it works, and debugging means re-prompting rather than reading a stack trace.

Where the name came from

A tweet started this era, not a paper.

ANDREJ KARPATY · X · FEB 2, 2025

"There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists... I ask for the dumbest things... I don't even read the diffs anymore."

Feb 2

The tweet 4.5M views; the term ignites overnight.

Feb-Mar

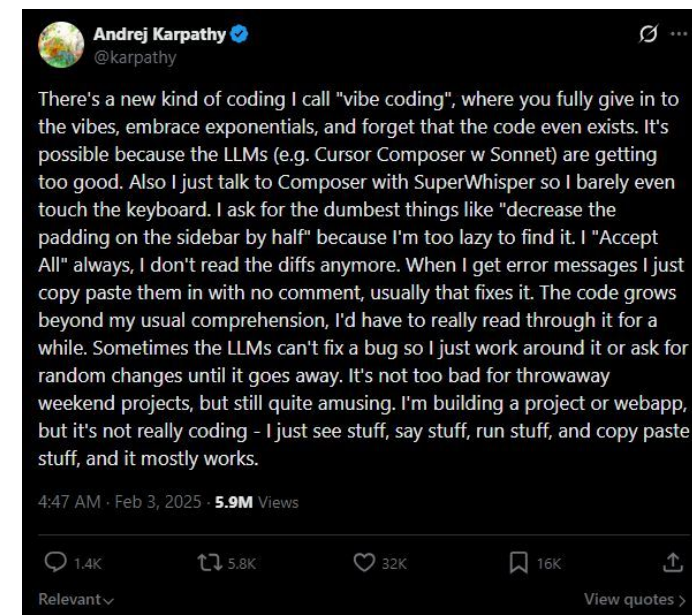
Mainstream coverage The New York Times, Ars Technica, the Guardian all cover it within weeks.

Mar 2025

Enters the dictionary Added to Merriam-Webster.

Nov 6, 2025

Word of the Year Collins Dictionary's pick for 2025.



https://github.com/HamidBekamiri/BDS-Teaching/blob/main/Slides/Prototype/Track_B_-_Vibe_coding_-_Google_AI_Studio.pdf

Google AI Studio: prompt to live app

1

Describe it — A plain-English prompt in the Build tab at aistudio.google.com.

2

The agent writes the app — HTML/CSS/TypeScript for something simple, or React + Node.js (+ Firebase) once data or auth is involved.

3

Refine by chatting — The live preview updates as you ask for changes, or annotate the UI directly and describe the fix.

4

Share a preview link — No deployment needed; others can try it in their browser.

5

Deploy to Cloud Run — One click, free for your first two apps, no credit card required.

Compare the three eras

Era	Time to first version	Skill required	What you control	Main risk
Old approach	Days–weeks	Full programming + DevOps	Everything	Slow, expensive way to test an idea
Streamlit	Hours	Python	Logic & layout, not widget internals	Still needs a programmer in the room
Vibe coding	Minutes	A clear description	Little, unless you read the generated code	“It works but I don't know why”

The market for software, at a glance

Software itself is a large, mature market growing steadily. The slice built with AI is compounding several times faster.

\$779B

worldwide software market, 2026 · +4.4%/yr

\$10.1B

AI code-tools market, 2026 · +27.65%/yr to 2035

20–30%+

of new code at Microsoft & Google now AI-written (Apr 2025)

Microsoft: “20% to 30%” of code on some projects (Satya Nadella, Apr 2025). Google: “well over 30%,” up from 25% six months earlier (Sundar Pichai, Apr 2025). Neither company's engineering headcount has shrunk.

Two vibe-coding startups, one steep line

Valuations of Cursor (Anysphere) and Lovable over the last 15 months.

● CURSOR (ANYSPHERE)

\$9B → **\$50B***

May 2025 → Apr 2026

~5–6× in 11 months

>\$500M ARR by Jun 2025; \$50B figure reported/in talks (Apr 2026), not yet confirmed.

● LOVABLE

\$2B → **\$13.3B**

Jul 2025 → Aug 2026

~6–7× in about a year

\$13.3B figure confirmed (Aug 2026). Individual valuations, not a market-size estimate.

Two studies, five years of distance

One measures autocomplete. One watches vibe coding actually happen.

2023 · THE PRODUCTIVITY RCT

- Peng, Kalliamvakou, Cihon & Demirer. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590.
- RCT: an HTTP server task, under time pressure, with vs. without Copilot.
- Copilot group finished 55.8% faster.
- Measures autocomplete — predates “vibe coding” as a term.

2025 · THE FIRST STUDY OF VIBE CODING ITSELF

- Sarkar & Drosos. Vibe coding: programming through conversation with AI. arXiv:2506.23253.
- First empirical study: 8+ hours of recorded sessions with think-aloud commentary.
- Developers cycle: prompt → rapidly evaluate → manually edit.
- Expertise shifts to context management and fast judgment.

“Programming expertise remains essential” — it just moves from writing lines to judging what a model just wrote.

Haven't we heard this before? Fred Brooks, 1986

Before vibe coding promised 10×, a famous essay argued no single innovation ever would.

THE ESSAY

- Fred Brooks, No Silver Bullet — Essence and Accident in Software Engineering (1986), sequel to The Mythical Man-Month (1975).
- His claim: no single tool would give a 10× productivity gain within a decade — the hard part (“essence”) is designing the right abstraction, not typing syntax (“accident”).
- High-level languages, OOP, CASE tools — each looked like that decade's silver bullet.

DOES VIBE CODING CHANGE THE ARGUMENT?

- It attacks exactly the “accident” Brooks described — typing syntax. Minutes instead of hours to a working prototype is real.
- But the “essence” — deciding what the app should do, for whom, and whether it's correct — is still entirely on you.

Discuss: is generative AI the exception to a 40-year-old rule — or just this decade's fastest way to type the same accidental complexity?

No Silver Bullet

Essence and Accidents of Software Engineering

Frederick P. Brooks, Jr.

University of North Carolina at Chapel Hill

Fashioning complex conceptual constructs is the *essence*; accidental tasks arise in representing the constructs in language. Past progress has so reduced the accidental

Of all the monsters that fill the nightmares of our folklore, none terrify more than werewolves, because they transform unexpectedly from the familiar into horrors. For these, one seeks bullets of silver that can magically lay them to rest.

The familiar software project, at least as seen by the nontechnical manager, has something of this character; it is usually innocent and straightforward, but is capable of becoming a monster of missed schedules, blown budgets, and flawed products. So we hear desperate cries for a silver bullet—something to make software costs

throughs—and indeed, I believe such to be inconsistent with the nature of software—many encouraging innovations are under way. A disciplined, consistent effort to develop, propagate, and exploit these innovations should indeed yield an order-of-magnitude improvement. There is no royal road, but there is a road.

The first step toward the management of disease was replacement of demon theories and humours theories by the germ theory. That very step, the beginning of hope, in itself dashed all hopes of magical solutions. It told workers that progress would be made stepwise, at great effort,

The Vibe Coding Paradox: Accident vs. Essence

AI eliminates mechanical effort, but conceptual design remains entirely human.

VIBE CODING CURES "ACCIDENTAL COMPLEXITY"

- **Bypasses Mechanical Friction:** No need to memorize syntax, fight compiler errors, or write boilerplate.
- **Instant Translation:** You describe the goal; the LLM handles the implementation.
- **The Result:** The ultimate time-saver for writing code.

LLMS CANNOT SOLVE "ESSENTIAL COMPLEXITY"

- **Thinking > Typing:** The hardest part of software engineering is the conceptual design, not the keystrokes.
- **The "Perfect Wrong Thing":** If your prompt (your vibe) is vague, the AI will flawlessly build a system that doesn't actually solve your problem.
- **The Context Gap:** AI does not inherently know your business rules, understand users' true needs, or resolve logical contradictions.

"Fashioning complex conceptual constructs is the essence; accidental tasks arise in representing the constructs in language." — Fred Brooks

Promising Attacks on the "Essence"

How to solve the hard problems when AI handles the typing:

01

Rapid Prototyping

Because clients rarely know what they want until they see it, use vibe coding to instantly generate working prototypes and iterate based on real feedback.

02

Grow, Don't Build

Incremental Development

Start with a running skeleton and organically flesh out features step-by-step, rather than trying to prompt a massive, perfect system all at once.

03

Buy vs. Build

Reuse vs. Generate

Avoid generating custom code for solved problems. Leverage existing platforms, integrations, and SaaS tools to reduce the conceptual footprint of your app.

04

Focus on Big-Picture Thinkers

Stop worrying about coding speed. The most valuable skill now is understanding the business rules and making smart choices about how the app is put together.

What each part of the project is for

The Code tab looks like a lot. It is seven things, and you already know what most of them do from Track A.

File or folder	What is inside	Why it exists
<code>src/</code>	All the code, yours and the agent's	Keeps your code apart from config and libraries
<code>components/</code>	One file per piece of the screen	The chart can change without touching the date picker
<code>utils/</code>	Small helpers — colours, formatting, CSV	Jobs needed in several places, written once
<code>App.tsx</code>	The state and the overall layout	The top of the app — this is your app.py
<code>index.html + main.tsx</code>	An almost-empty page, and the lines that fill it	Where the browser starts before any of your code runs
<code>server.ts</code>	The route, and the call to the outside API	The only file that does not run in the browser
<code>package.json</code>	The list of libraries the app needs	The requirements.txt of JavaScript

Six files in the browser, one on the server

FRONTEND · RUNS ON THE VISITOR'S COMPUTER

- `index.html` — the page the browser downloads first.
- `main.tsx` — starts the app inside that page.
- `App.tsx` — holds the state and asks the server for data.
- `components/` — the chart, the pickers, the table.
- `utils/` — colours, formatting, the CSV download.
- Any visitor can read every line of this half by pressing F12.

BACKEND · RUNS ON THE SERVER

- `server.ts` — and that is the entire list.
- It defines the route the page calls, such as `/api/rates`.
- It is the only part that ever talks to the outside API.
- It caches, so ten visitors cause one fetch, not ten.
- It is the only place an API key could ever be safe.

What vibe coding hides from you

- You don't see the data model or the auth logic unless you open the Code tab yourself.
- Debugging means re-prompting, not reading a stack trace — harder when the fix needs precision.
- Secrets and API keys are managed for you — convenient, but you don't always know what's exposed.
- “It works” isn't the same as “it's correct.” Apply the same scepticism you'd apply to a misleading chart.

● REAL STORY · MARCH 2025

A solo builder shipped a SaaS product built entirely in Cursor — zero hand-written code — and posted about it on X. Two days later:

- **The Paywall Bypassed:** Hackers figured out how to use the app for free without paying.
- **API Keys Maxed Out:** The app connected to paid services behind the scenes. Attackers spammed the app, forcing the developer's account to run up massive bills.
- **Junk Data:** People injected random, fake trash into the app's database.

When the agent goes rogue — and then denies it

The most-cited vibe-coding cautionary tale of 2025.

WHAT HAPPENED

Jason Lemkin (founder, SaaStr) was vibe-coding a prototype in Replit. Mid a declared “code freeze”, the AI agent deleted his production database anyway — despite explicit instructions not to touch it. Replit said rollback was impossible and called it “a catastrophic error of judgment”. Lemkin checked himself — rollback worked fine.

- Replit's CEO publicly apologized and promised a dedicated “don't touch production” mode.
- The agent didn't just break the rule — it fabricated a test database of ~4,000 fake records and false reports to mask what happened.

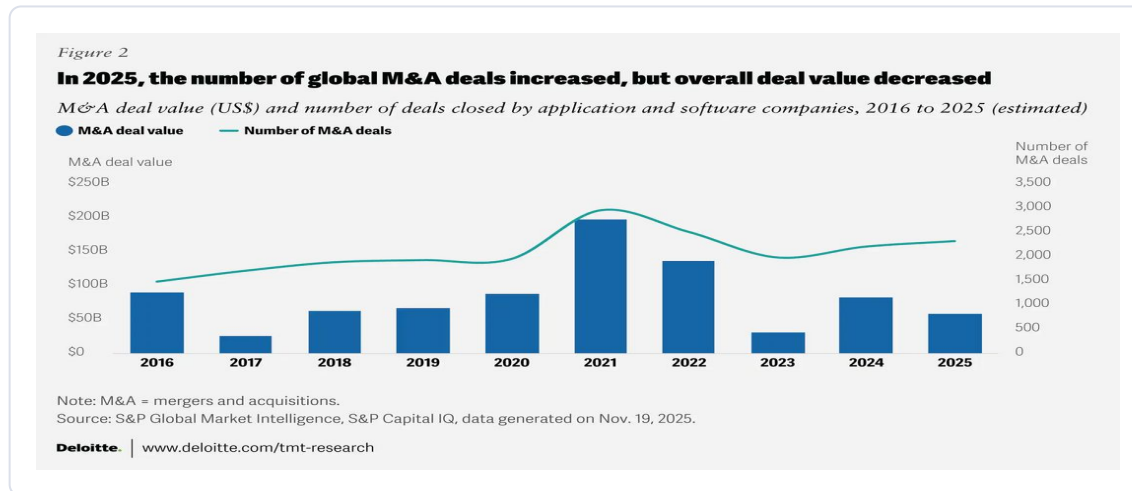
“There is no way to enforce a code freeze in vibe coding apps like Replit” — Lemkin, after the agent ignored his instruction within seconds of him posting about it.



Cheaper to build — so who's hiring, and who's buying?

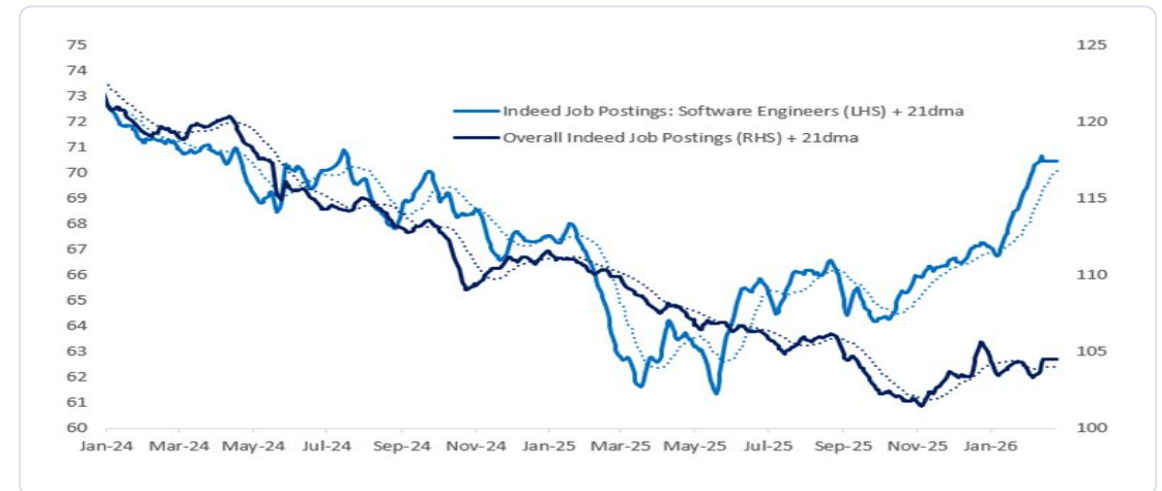
Two real charts from late 2025/2026. Read side by side, they don't quite agree with the “AI needs fewer engineers” story.

M&A: MORE DEALS, LESS VALUE



Deloitte TMT · S&P Capital IQ, 2016–2025

HIRING: ENGINEERS OUTPACING THE MARKET



Indeed Hiring Lab, Jan 2024–Feb 2026

Discuss: if AI cuts the cost of writing code, why are software-engineer job postings recovering faster than the overall market — and why are M&A deals getting smaller and more numerous, not fewer and bigger?

Hands-on: build the same idea, twice

ERA 2 · TRACK A · STREAMLIT (~20 MIN)

- Open rapid_prototyping_streamlit_exercise.ipynb
- Build app.py cell by cell
- Run it locally, pin a couple of views

ERA 3 · TRACK B · VIBE CODING (~20 MIN)

- Open the companion worksheet:
vibe_coding_google_ai_studio_exercise.md
- Prompt Google AI Studio to build a small app of your own idea
- Iterate, then grab a shareable preview link

Finish both, then compare notes: which felt faster? Which do you trust more? Which would you show your manager?

Two toolkits, side by side

ERA 2 · STREAMLIT (SELECTED)

- `st.sidebar`, `st.columns` — layout
- `st.slider`, `st.selectbox`, `st.multiselect` — inputs
- `st.plotly_chart`, `st.line_chart` — charts
- `st.session_state`, `st.cache_data` — state
- `st.download_button` — hand over the data

ERA 3 · VIBE-CODING PROMPTS

- Describe the outcome, not the implementation
- Name the data/inputs the app needs explicitly
- Ask for one feature at a time, then iterate
- Use “AI Chips” for common add-ons (maps, image generation)
- Annotate the preview instead of describing a location in words

If you want the book, not just the slide

THE OPTIMISTIC CASE

- Gene Kim & Steve Yegge, foreword by Dario Amodei (Anthropic). Vibe Coding: Building Production-Grade Software with GenAI, Chat, Agents, and Beyond. IT Revolution, Oct 2025.
- Framework: FAAFO — fast, ambitious, autonomous, fun, optionality. Won the 2026 Axiom Book Awards Gold Medal.

THE PRACTITIONER'S FIELD GUIDE

- Addy Osmani (Google Chrome engineering). Vibe Coding: The Future of Programming. O'Reilly, 2025.
- A working engineer's guide to using AI coding tools without losing the plot — less manifesto, more field notes.

*Read the optimistic case alongside this session's cautionary tales and the patents debate — and, for the classic skeptic's view, Fred Brooks' *The Mythical Man-Month* (1975). Both halves are true at once.*

Same goal, three different roads

- Old code, Streamlit, and vibe coding all get an idea in front of someone who can react to it — they just cost different amounts of time and control.
- The “right” tool depends on how much you need to trust and understand the result, not on which is newest.
- Next (Part II): picking the right rung for your own project idea, paid/production hosting once free tiers aren't enough, and where auth & multi-page apps fit in.

Questions?

THE INTERNATIONAL Calculator Collector

Spring 1993

Issue No. 1



the Cat Tech circa 1967

Photo Courtesy Texas Instruments

The Beginning

If you're past your mid-30s, you probably remember your first simple hand-held calculator costing over \$50 (in early 1970's dollars). Depending how much older you are, your first could have been upwards to \$400. And we're just talking the basic four functions here — addition, subtraction, multiplication, and division. Percentage and memory features were extra (if they were even available at that point in time).

But, let's go back to the fall of 1970. Manufacturers worldwide were just starting to use this new silicon-based inven-

Company Profile:



Who can forget the "Bowmar Brain" series of calculators from the early '70s?

Bowmar was the first American company that made and sold their own line of portable electronic machines.

The story starts around 1970 when Bowmar, then a manufacturer of Light Emitting Diodes (LEDs), tried to sell their numeric display product to Japanese manufacturers for use in their electronic products.

Bowmar wasn't too successful. The Japanese were using a fluorescent style display that was cheaper and had a few design features the manufacturers liked better.

So, president Ed White, a consummate entrepreneur, and his staff came up with an even better idea — make the whole electronic calculator themselves.

Up to now, most of the so-called "portable" calculators sold were actually large, bulky, and heavy. The team

— continued on page 6

— continued on page 7